

**** LAB ** Gestione di processi [17 marzo]**

Sito: [Virtuale](#)

Corso: Amministrazione di Sistemi T - Laboratorio di
Amministrazione di Sistemi T

Libro: ** LAB ** Gestione di processi [17 marzo]

Stampato da: Lorenzo Bernardini

Data: venerdì, 24 aprile 2026, 11:53

Sommario

1. Processi in background
2. Soluzione esercizio
3. nohup e disown
4. array (un esempio)
5. wait e watch (due comandi non correlati ma utili)
6. Esercizio: comandi in parallelo da sincronizzare
7. Soluzione: comandi in parallelo da sincronizzare

1. Processi in background

Lanciare i comandi elencati, commentando quel che ne risulta

```
sleep 1000 &
```

```
A=$!
```

```
echo $A
```

```
ps $A
```

```
jobs
```

```
jobs -l
```

```
kill -STOP $A
```

```
ps $A
```

```
jobs
```

```
kill -CONT $A
```

```
ps $A
```

```
jobs
```

```
kill $A
```

```
ps $A
```

```
jobs
```

Esercizio: si realizzi uno script che lancia i comandi **sleep 10** e **sleep 20** in parallelo, e che ogni 5 secondi scriva su STDOUT il loro stato (in esecuzione / terminato)

2. Soluzione esercizio

```
#!/bin/bash

cat /dev/null > log

sleep 10 &
PRIMO=$!

sleep 20 &
SECONDO=$!

while sleep 5 ; do

    STAT01=`ps hp $PRIMO | awk '{ print $3 }'`
    STAT02=`ps hp $SECONDO | awk '{ print $3 }'`

    if test -z "$STAT01" ; then STAT01=terminato ; fi
    if test -z "$STAT02" ; then STAT02=terminato ; fi

    echo "Stato del processo $PRIMO: $STAT01, stato del processo $SECONDO: $STAT02" | tee -a log

    # if test "$STAT01" = "terminato" -a "$STAT02" = "terminato" ; then break ; fi
    if ! ps "$PRIMO" && ! ps "$SECONDO" ; then break ; fi
done
```

3. nohup e disown

Un processo in background resta legato alla shell che lo ha avviato.

Verificate

- col comando ps
- col comando jobs
- chiudendo la shell

qual è l'effetto di lanciare un job in questi modi

- sleep 1000 &
- sleep 1000 & disown
- nohup sleep 1000 &

4. array (un esempio)

In bash è possibile usare array con indici non consecutivi.

Si può quindi utilizzare un array per conservare informazioni su di un processo, indicizzandolo col PID.

Esempio (al solito, si ragioni preventivamente su quel che fanno i comandi, e poi sul loro risultato):

```
COMANDO1="sleep 10"
```

```
COMANDO2="sleep 20"
```

```
$COMANDO1 & PID[$!]=$COMANDO1
```

```
$COMANDO2 & PID[$!]=$COMANDO2
```

```
echo ${!PID[@]}
```

```
echo ${PID[@]}
```

```
# ora provate a estrarli singolarmente
```

5. wait e watch (due comandi non correlati ma utili)

Aprirete due terminali sulla vm (con **vagrant ssh**)

in uno lanciate

```
sleep 30 &
```

```
sleep 40 &
```

```
wait
```

nell'altro

```
watch "ps auxw | grep sleep | grep -v grep"
```

Notate (nel primo terminale) come wait attenda la terminazione di tutti i job: utile per la sincronizzazione.

Notate (nel secondo terminale) l'utilità di watch per osservare periodicamente l'output di un comando.

6. Esercizio: comandi in parallelo da sincronizzare

Calcolare in parallelo il numero di file esistenti in due sottoalberi originati da due directory passate come parametri.

Stampare come risultato la frase "la directory ... contiene piu' file della directory ..."

Nota: i due processi di conteggio vanno in background, ma devo aspettare la terminazione di entrambi per poter confrontare i risultati!

Realizzate lo script gestendo il controllo dei parametri.

7. Soluzione: comandi in parallelo da sincronizzare

```
#!/bin/bash
```

```
function testd() {  
    test -d "$1" && return 0  
    #si potrebbe essere più precisi testando diversi tipi di errore  
    #es. non esiste, esiste ma non è directory, ...  
}
```

```
    echo "Il parametro non è una directory"  
    exit 1  
}
```

```
function conta() {  
    find "$1" -type f 2>/dev/null | wc -l  
}
```

```
testd "$1"  
testd "$2"
```

```
FA=$(mktemp)  
FB=$(mktemp)  
# file temporanei: sano usare il tool che li crea senza rischi di  
# omonimia o altri problemi di sicurezza es. corse critiche
```

```
conta "$1" > "$FA" &  
conta "$2" > "$FB" &  
# notare la necessità di due file diversi: processi concorrenti  
# non possono scrivere sullo stesso file senza interferenza  
# a meno di implementare complessi meccanismi di sincronizzazione  
# ... ma il filesystem lo fa gratis
```

```
wait
```

```
CA=$(cat "$FA")  
CB=$(cat "$FB")
```

```
if [[ $CA -gt $CB ]] ; then  
    echo "la directory $1 contiene piu' file ($CA) della directory $2 ($CB)"  
elif [[ $CA -lt $CB ]] ; then  
    echo "la directory $2 ($CB) contiene piu' file della directory $1 ($CA)"  
else  
    echo "le directory $1 e $2 contengono lo stesso numero di file ($CA)"  
fi
```

```
rm -f "$FA" "$FB"  
# importante fare pulizia
```